Laboratorium 4 — Funkcje skrótu

3 zadania po 1 pkt. Zajęcia: 2 × 45 minut. Plan: 10 minut wprowadzenia, około 70 minut pracy z kodem (20 + 25 + 25 minut) i 10 minut na omówienie oraz rezerwę. Napisz cały kod samodzielnie w **dowolnym języku**. Użyj biblioteki do SHA-256, PBKDF2-HMAC-SHA256, porównania w stałym czasie i losowania bajtów; nie implementuj tych algorytmów od podstaw. Wystarczy prosty program bez menu. Raport i porządkowanie repozytorium możesz dokończyć po zajęciach.

Dozwolone: ISOD, dokumentacja, własne notatki i własny wcześniej napisany kod. Podczas zajęć wyłącz czaty i podpowiedzi LLM. Przy odbiorze wyjaśniasz działanie programu.

Wszystkie pliki czytaj **binarnie** i nie zmieniaj końców wierszy (pliki mają LF). Skrót dotyczy dokładnych bajtów pliku.

Zadanie 1 — Sumy kontrolne (około 20 min, 1 pkt)

Plik `SHA256SUMS.txt` ma format programu `sha256sum`: w każdym wierszu 64 znaki hex skrótu SHA-256, dwie spacje i ścieżka pliku względem katalogu, w którym leży `SHA256SUMS.txt`.

1. Napisz funkcję liczącą SHA-256 pliku. Możesz wczytać cały plik naraz.
2. Dla każdego wiersza `SHA256SUMS.txt` wypisz ścieżkę i status: `OK` (skrót zgodny), `BŁĄD` (inny skrót) albo `BRAK` (pliku nie ma). Brakujący plik nie może przerwać programu.
3. Oblicz SHA-256 tekstów ASCII `Ala ma kota.` i `Ola ma kota.`. Policz, ile z 256 bitów różni się w obu skrótach (XOR skrótów i liczba jedynek).

Samokontrola: na Linuksie lub w Git Bash możesz porównać statusy z wynikiem `sha256sum -c SHA256SUMS.txt`.

W raporcie: status każdego pliku, liczba różniących się bitów oraz jedno zdanie: dlaczego suma kontrolna zapisana obok pliku nie chroni przed celową podmianą i czego trzeba użyć zamiast niej.

Zadanie 2 — Przechowywanie haseł (około 25 min, 1 pkt)

Plik `hasla.txt` zawiera rekordy `login:iteracje:sól_hex:skrót_hex`. Skrót to **PBKDF2-HMAC-SHA256** z bajtów UTF-8 hasła, z solą 16 B i liczbą iteracji z rekordu; wynik ma 32 B.

1. Wczytaj bazę i napisz `weryfikuj(login, hasło)`: wylicz skrót z parametrami rekordu i porównaj go z zapisanym **funkcją porównania w stałym czasie** z biblioteki. Nieznany login daje fałsz. Sprawdź cztery przypadki:
   - `anna`, `Kasztan-2026` → prawda;
   - `anna`, `kasztan-2026` → fałsz;
   - `piotr`, `Żółw na łące` → prawda (napis wpisz w kodzie źródłowym zapisanym w UTF-8);
   - `kasia`, `Kasztan-2026` → fałsz.
2. Napisz `dodaj(login, hasło)`: sól 16 B z bezpiecznego generatora, 600000 iteracji, rekord w tym samym formacie. Dodaj `test1` i `test2` z tym samym hasłem, porównaj ich sole i skróty oraz zweryfikuj oba konta. Zapis do pliku nie jest wymagany.

W raporcie: wyniki czterech testów, wynik porównania soli i skrótów nowych kont, wynik weryfikacji obu kont oraz jedno zdanie: co daje sól, a co liczba iteracji. Nie przepisuj pełnych rekordów.

Zadanie 3 — Kolizje i paradoks urodzinowy (około 25 min, 1 pkt)

Model krótkiego skrótu: **h24(dane) = pierwsze 3 bajty SHA-256(dane)**, czyli 6 pierwszych znaków hex. Pliki `oryginal.txt` i `zmieniony.txt` mają po 13 wierszy zakończonych bajtem `\n` i różnią się treścią.

Wariant v dokumentu, dla 0 ≤ v < 8192: dla każdego wiersza i = 0…12, licząc od góry, dopisz jedną spację przed jego `\n`, jeśli `(v >> i) & 1` wynosi 1. Pozostałe bajty zostają bez zmian. Wariant 0 to plik bez zmian.

1. Policz h24 wszystkich 8192 wariantów oryginału i zapamiętaj je w słowniku h24 → v. Gdy skrót się powtórzy, wystarczy zachować jeden numer wariantu.
2. Przeglądaj warianty zmienionego dokumentu i znajdź **jedną parę** (v_oryginał, v_zmieniony) o równym h24. Wypisz parę i jej h24. Możesz zakończyć szukanie pary po pierwszym trafieniu; dowolna poprawna para jest akceptowana. Nie trzeba sortować par ani zapisywać wariantów do plików.
3. Sprawdź wszystkie 8192 warianty zmienionego dokumentu i policz, ile ma h24 równy h24 niezmienionego oryginału (wariant 0). Możesz wykorzystać skróty z kroku 2 i kontynuować tę samą pętlę do końca.

W raporcie: znaleziona para z h24, wynik kroku 3 oraz jedno zdanie: dlaczego znalezienie dowolnej pary jest znacznie łatwiejsze niż dopasowanie do ustalonego skrótu.

Dodatkowo — dla chętnych, poza zakresem na 90 minut

Te ćwiczenia nie są wymagane do uzyskania pełnych 3 pkt ani do raportu.

- Licz SHA-256 pliku w kawałkach po 4096 B. Dla `dokumenty/dane.bin` porównaj wynik ze skrótem całego pliku wczytanego naraz.

- Wypisz loginy z mniej niż 600000 iteracji. Zmierz średni czas PBKDF2 dla 1 iteracji (co najmniej 10000 wywołań) i 600000 iteracji (5 wywołań). Oszacuj czas sprawdzenia wszystkich 26^6 = 308 915 776 haseł dla jednego rekordu: liczba kandydatów × czas jednej próby.

- Znajdź wszystkie pary kolizyjne, przechowując listę wariantów dla każdego h24. Porównaj liczbę par z wartością oczekiwaną $8192 \cdot 8192 / 2^{24}$.

Oddanie — tylko jeden plik raportu

Oddaj **jeden plik TXT, PDF albo MD**: `nazwisko_nrindeksu_lab04` z odpowiednim rozszerzeniem. Raport do jednej strony powinien zawierać imię i nazwisko, numer indeksu, wybrany język, **link do repozytorium GitHub i identyfikator commitu** oraz wyniki i krótkie odpowiedzi wskazane w trzech zadaniach.

Cały samodzielnie napisany kod umieść na GitHubie. Dodaj krótki README z wymaganymi bibliotekami i komendą uruchomienia. Repozytorium musi być dostępne dla prowadzącego (publiczne albo prywatne z nadanym dostępem). Raport i porządkowanie repozytorium możesz dokończyć po zajęciach, do terminu podanego przez prowadzącego. **Nie oddawaj ZIP-a, kodu ani dodatkowych załączników.** Miejsce oddania raportu poda prowadzący.

Punktacja

Za każde zadanie: **0,50 pkt** za działający, samodzielnie napisany kod na GitHubie; **0,25 pkt** za wymagane wyniki w raporcie; **0,25 pkt** za poprawne wyjaśnienie działania. Łącznie **3 pkt**. Wybór języka nie wpływa na ocenę.